

COMPANION
to

ChatGPT Teaches TikZ

*Chapter 22: Reusable Pictures with **pic** (A)*

Contents

22.1	What This Companion Adds	2
22.2	A Pic Vocabulary	3
22.3	Your First Pic	4
22.4	Placing Several Instances	4
22.5	Allowing Pic Actions	5
22.6	Combining Pics and Semantic Styles	5
22.7	Exposing Connection Coordinates	5
22.8	Transforming an Instance	7
22.9	Choosing the Component Boundary	7
22.10	Worked Project: Reusable Modules	7
22.11	Debugging Workshop	9
22.12	Exercises	10
22.13	Solutions	10
22.14	ChatGPT Workshop	12
22.15	Final Checklist	12

22.1 What This Companion Adds

Chapter 22 introduced a `pic` as a reusable little drawing. This Companion develops a modular design method: place geometry inside the `pic`, keep appearance in semantic styles, expose only useful connection coordinates, and transform instances rather than copying commands.

Design

Use a style to reuse appearance. Use a pic to reuse a coherent group of drawing commands and internal coordinates.

22.2 A Pic Vocabulary

Form or option	Purpose
<code>pics/name/.style=</code>	Defines a reusable pic.
<code>code={...}</code>	Contains the pic's drawing commands.
<code>\pic {name}</code>	Places one instance.
<code>\pic at (x,y)</code>	Places an instance at a coordinate.
<code>pic actions</code>	Applies options supplied to the pic's paths.
<code>\pic (A)</code>	Assigns a prefix name to one instance.
<code>(-left)</code>	Defines a named coordinate exposed by the pic.
<code>rotate, scale</code>	Transform an instance at placement time.

22.3 Your First Pic

```
\tikzset{
  pics/target/.style={
    code={
      \draw (0,0) circle (5mm);
      \draw (0,0) circle (2.5mm);
      \fill (0,0) circle (1.5pt);
    }
  }
}
```

Place it with one command.

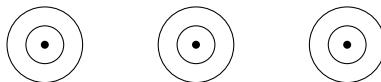
```
\pic {target};
```

The coordinate system inside the pic is local to its geometry.

22.4 Placing Several Instances

```
\pic at (0,0) {target};
\pic at (2,0) {target};
\pic at (4,0) {target};
```

See



Only placement varies. A change inside the definition updates all three instances.

22.5 Allowing Pic Actions

Hard-coded path options make a pic less adaptable. The special style `pic actions` receives options supplied at placement time.

```
\tikzset{
  pics/target/.style={
    code={
      \draw[pic actions] (0,0) circle (5mm);
      \draw[pic actions] (0,0) circle (2.5mm);
    }
  }
}
\pic[blue,thick] {target};
```

Use this mechanism only for properties that should genuinely be controlled by the surrounding picture.

22.6 Combining Pics and Semantic Styles

```
\tikzset{
  construction/.style={densely dashed,thin},
  point/.style={draw,circle,fill=white,inner sep=1.5pt},
  pics/midpoint mark/.style={
    code={
      \draw[construction] (-.45,0)--(.45,0);
      \node[point] at (0,0) {};
    }
  }
}
```

The pic owns the geometry; the project styles own the appearance.

22.7 Exposing Connection Coordinates

```

\tikzset{
  pics/module/.style={
    code={
      \draw[rounded corners]
        (-.8,-.5) rectangle (.8,.5);
      \coordinate (-left) at (-.8,0);
      \coordinate (-right) at (.8,0);
    }
  }
}

```

Assign names to two instances and connect their exposed coordinates.

```

\pic (A) at (0,0) {module};
\pic (B) at (3,0) {module};
\draw (A-right)--(B-left);

```

The larger picture need not know the internal coordinates.

Common Mistake

Prefix names inside a pic with a hyphen when they should combine with the instance name. Otherwise repeated instances can create duplicate node names.

22.8 Transforming an Instance

```
\pic[rotate=30] at (0,0) {module};  
\pic[scale=.8] at (3,0) {module};
```

The pic's internal commands remain unchanged. Use **transform shape** as well when node shapes and text should follow a geometric transformation.

22.9 Choosing the Component Boundary

A good pic represents one recognizable unit: a module, marker, gate, symbol, or small construction. Do not place an entire unrelated figure inside one pic merely to hide source code. The boundary should match a concept that the larger diagram can place and connect.

22.10 Worked Project: Reusable Modules

22.10.1 Stage 1: Define Styles and Geometry

```
\tikzset{  
  module box/.style={  
    draw,rounded corners,minimum width=22mm,  
    minimum height=10mm  
  },  
  connection/.style={-{Stealth},thick},  
  pics/module/.style={code={  
    \node[module box] (-box) at (0,0) {};  
    \coordinate (-left) at (-1.1,0);  
    \coordinate (-right) at (1.1,0);  
  }}  
}
```

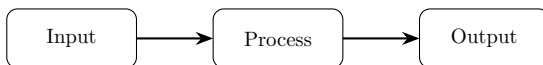
22.10.2 Stage 2: Place and Connect Instances

```
\pic (A) at (0,0) {module};  
\pic (B) at (3.5,0) {module};  
\pic (C) at (7,0) {module};  
\draw[connection] (A-right)--(B-left);  
\draw[connection] (B-right)--(C-left);
```


22.10.3 The Complete Picture

```
\begin{tikzpicture}
  \pic (A) at (0,0) {module};
  \pic (B) at (3.5,0) {module};
  \pic (C) at (7,0) {module};
  \node at (A-box) {Input};
  \node at (B-box) {Process};
  \node at (C-box) {Output};
  \draw[connection] (A-right)--(B-left);
  \draw[connection] (B-right)--(C-left);
\end{tikzpicture}
```

See



22.11 Debugging Workshop

22.11.1 Repeated Instances Share a Name

Give each `pic` instance a distinct name and use hyphen-prefixed internal names.

22.11.2 Placement Options Do Not Affect Paths

Use `pic` actions in the intended internal drawing commands.

22.11.3 A Connection Knows Internal Geometry

Expose semantic coordinates such as `-left` and `-right` instead of repeating internal numerical positions outside the pic.

22.11.4 The Pic Is Too Large Conceptually

Split it into smaller components when its parts are placed or reused independently.

22.12 Exercises

1. Define a pic containing two concentric circles.
2. Place the pic at three coordinates.
3. Make its path appearance respond to `pic actions`.
4. Combine one pic with two semantic styles.
5. Add two exposed connection coordinates.
6. Name and connect three instances.
7. Rotate one instance without changing the definition.
8. Convert a repeated component from an earlier figure into a pic.

22.13 Solutions

22.13.1 Exercises 1–4

```
\tikzset{pics/rings/.style={code={  
  \draw[pic actions] (0,0) circle (5mm);  
  \draw[pic actions] (0,0) circle (2.5mm);  
}}}  
\pic[blue] at (0,0) {rings};  
\pic[red] at (2,0) {rings};  
\pic[gray] at (4,0) {rings};
```

22.13.2 Exercises 5–7

```
\coordinate (-in) at (-.8,0);  
\coordinate (-out) at (.8,0);  
\pic (A) at (0,0) {module};  
\pic (B) at (3,0) {module};  
\draw (A-out)--(B-in);
```

Exercise 7 adds `rotate` to one placement. Exercise 8 moves repeated geometry into the `pic` while leaving surrounding connections outside.

22.14 ChatGPT Workshop

Ask ChatGPT to find the correct abstraction boundary for reusable drawings.

1. Identify repeated command groups that should become pics rather than styles.
2. Separate the geometry of this component from its semantic styles.
3. Design a minimal set of named coordinates for external connections.
4. Diagnose duplicate names created by several pic instances.
5. Refactor this pic so placement options affect only intended paths.

22.15 Final Checklist

Before continuing to Chapter 23, check that you can

1. distinguish reusable appearance from reusable geometry;
2. define and place a pic;
3. place several transformed instances;
4. combine pics with semantic styles;
5. use `pic actions` selectively;
6. expose named connection coordinates; and
7. keep internal coordinates private.